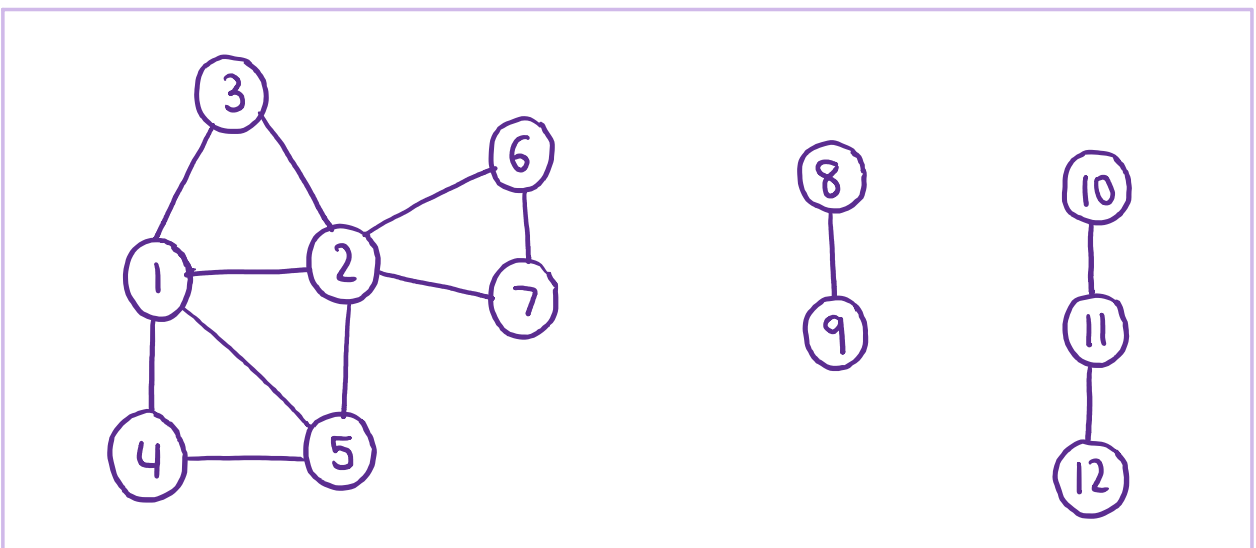


• DEPTH FIRST SEARCH

Init Discovered
 $R \leftarrow \{ \}$
DFS(s):
 Discovered(s) $\leftarrow$ True
 $R \leftarrow \{s\} \cup R$ ↘ $start[s] \leftarrow$ current time
 for each edge (s,u) incident on s :
 if Discovered(u) == False:
 DFS(u)
 End(s) $\leftarrow$ current time

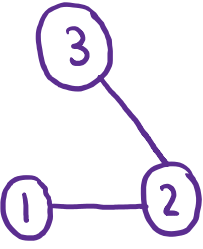
DFS(1):
 $R = \{1\}$
 $N(1) = \{2,3,4,5\}$



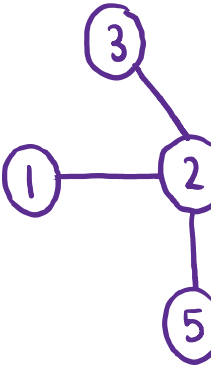
DFS(2):
 $R = \{1,2\}$
 $N(2) = \{1,3,5,6,7\}$



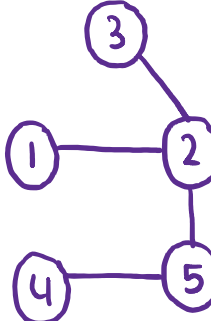
DFS(3):
 $R = \{1,2,3\}$
 $N(3) = \{1,2\}$



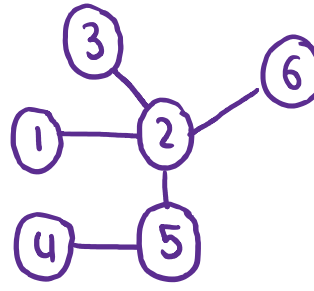
DFS(5):
 $R = \{1,2,3,5\}$
 $N(5) = \{1,2,4\}$



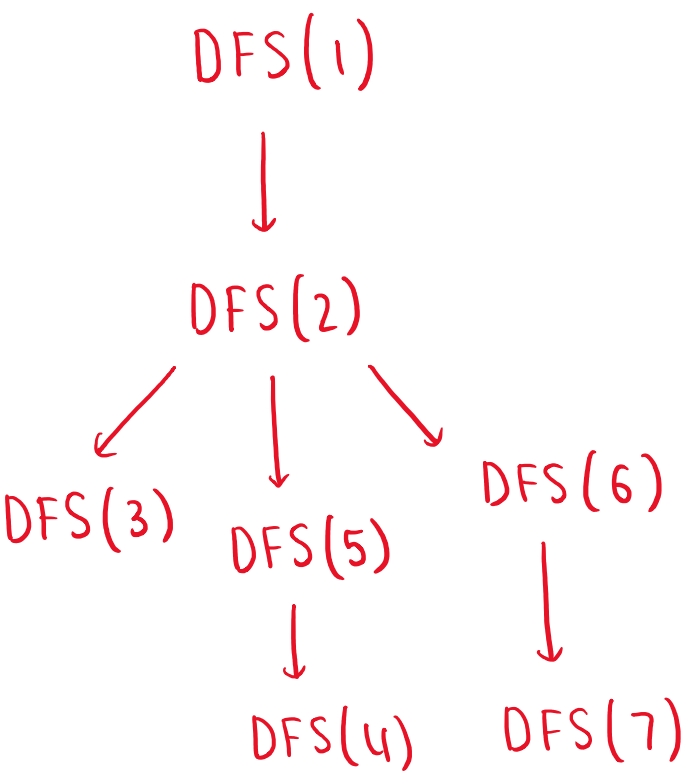
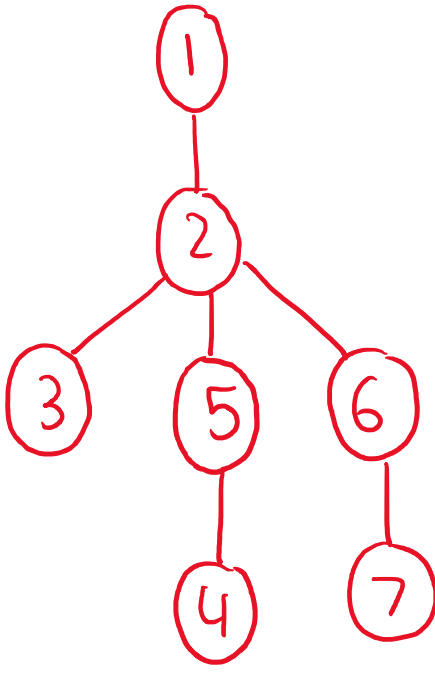
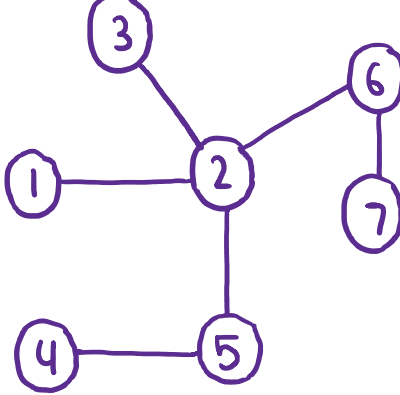
DFS(4):
 $R = \{1,2,3,5,4\}$
 $N(4) = \{1,5\}$



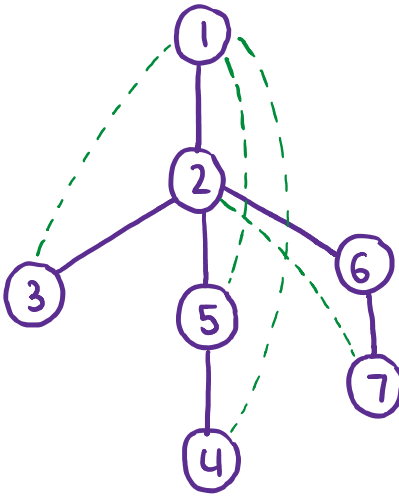
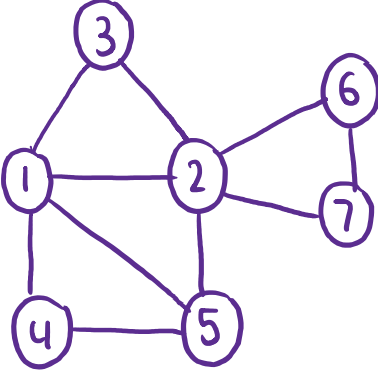
DFS(6):
 $R = \{1,2,3,4,5,6\}$
 $N(6) = \{2,7\}$



DFS(7):
 $R = \{1,2,3,4,5,6,7\}$
 $N(7) = \{2,6\}$



REMARK: Let us call the edges from E that appear in the DFS tree as tree edges.



All missing edges (non-tree edges) are from nodes to ancestors.
 $\Rightarrow$ no cross edges. ↘ BACK EDGES
 $\Rightarrow$ can be used to find cycles. If there are back edges $\Rightarrow$ not acyclic

OBSERVATION: No cross edges are possible.
We return back to parent node only after checking all neighbors of current node.

Q: What if the BFS tree and DFS tree are identical?
What can we say about the graph?

ans: This happens iff the graph is a tree.
BFS: look at all neighbors
DFS: look at one neighbor at a time

OBSERVATIONS:
• If u is a descendant of v in DFS tree,
 $start(v) < start(u) < end(u) < end(v)$
• If u and v are unrelated (on diff. branches),
 then $(start(u), end(u))$ and $(start(v), end(v))$ are disjoint.

• $start(u) < start(v) < end(u) < end(v)$
This cannot happen for any pair of vertices (u,v) .

• Between start and end times of a node u , lie the start-end intervals of all nodes reachable from u without going through parent(u).

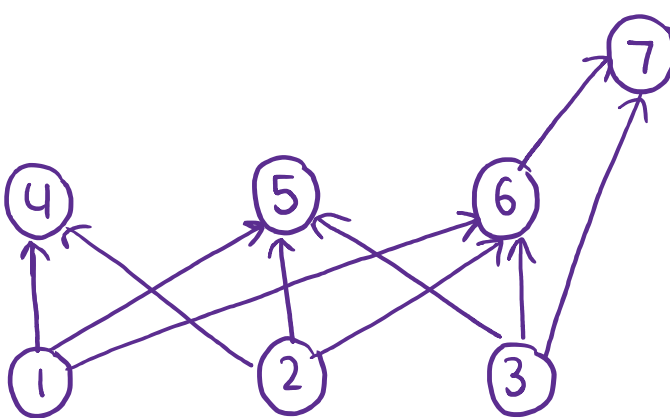
Consider a directed graph. (acyclic)
We want a topological sort.

Ordering of vertices:
• For any edge $(u,v) \in E$, there is an ordering $u \leq v$.
• Ordering is transitive
 $u \leq v, v \leq w \Rightarrow u \leq w$

We want the sorting as per $\leq$.

• Start with nodes having no incoming edges.

Topological sort (G):
• Initialise $InDegree[v] \forall v \in V(G)$.
• While $\exists$ a vertex that is not pushed into a DS:
 $U \leftarrow$ set of vertices with indegree 0
 For all $v \in N_{out}(U)$:
 $InDegree(v) = InDegree(v) - |N_{in}(v) \cap U|$
 DS.append(U).
 ↘ outgoing neighbors ↘ incoming neighbors



basically removing edges coming from vertices of indegree 0.

$U = \{1,2,3\}$
 $N_{out}(U) = \{4,5,6,7\}$

$U = \{4,5,6\}$
 $N_{out}(U) = \{7\}$

$InDegree(4) = 2 - 2 = 0$
 $InDegree(5) = 3 - 3 = 0$
 $InDegree(6) = 3 - 3 = 0$
 $InDegree(7) = 2 - 1 = 1$

$InDegree(7) = 1 - 1 = 0$
DS.append($\{7\}$)

DS.append($\{1,2,3\}$)

• Topological sort in a DFS tree is given by decreasing order of end times.

